

Scalability Plan: Handling 100,000 Concurrent Users Email Approval Automation System

Scalability Goal & Assumptions: To build a system that can sustain/support up to 100,000 users, assuming high read traffic and asynchronous processing for resource-intensive tasks such as AI analysis and workflow execution.

Overview: This document outlines how the system can scale to support up to 100,000 concurrent users. The architecture is designed to distribute workload across frontend, backend, database, workflow automation, and AI services to prevent bottlenecks and ensure reliability under high traffic.

1. Frontend Scalability

The frontend is deployed on Vercel.

- Traffic is spread out so no single server gets overloaded
- The hosting platform automatically handles more users
- Use Lighthouse audits to monitor performance, accessibility, and best practices
 - Helps identify slow load times and improve performance scores
- Implement lazy loading so only important content loads first (images/components load later)
 - Reduces initial load time and bandwidth usage

Result: Handles thousands of users without performance degradation.

2. Backend Scalability (API Layer)

The backend (FastAPI) will scale using:

- Use horizontal scaling by running multiple backend instances instead of relying on one server
 - Improves availability and fault tolerance
- Use load balancing (round-robin / least-connections) to distribute requests evenly across servers
- Use auto-scaling to add or remove backend instances based on traffic (CPU usage, request rate, latency)
- Use stateless API design
 - Any server can handle any request without relying on stored data
- Use efficient endpoints
 - return only necessary data
- Split requests across servers to avoid overload
- Use rate limiting to prevent excessive requests from a single user
 - Helps prevent abuse and API flooding

Result: Prevents server overload during traffic spikes.

3. Database Scalability (Supabase)

To handle large numbers of requests:

- Use indexing on frequent queries for faster lookups (status, timestamps, user_id)
 - Reduces full table scans and speeds up lookups
- Use read replicas for high read traffic
- Use connection pooling so multiple backend instances share database connections efficiently
- Use read replicas to handle heavy read traffic and reduce load on the main database
- Apply query limits/pagination to reduce load
 - Prevents large data fetches
- Optimize schema for frequent operations (email requests, decisions)
- Avoid unnecessary joins/heavy queries

Result: Faster data access and reduced database bottlenecks.

4. Workflow Scalability (n8n Automation)

n8n workflows are designed to run asynchronously:

- Use queue-based execution for processing requests (FIFO queue)
 - Ensures requests are processed in order without overload
- Limit concurrent executions to avoid overload to prevent system overload and ensure stable processing under high traffic (backpressure)
- Add retry logic for failed steps
 - Helps recover from temporary API/database failures
- Schedule workflows instead of running everything instantly
- Schedule non-urgent workflows (batch processing every few minutes instead of instantly)

Result: Prevents workflow failures when many requests are triggered.

5. AI API Scaling (Gemini)

- Only call AI when necessary (not for every request)
- Use AI response caching for repeated or similar requests
 - if a room is already booked and the AI suggests an alternative, store that result and reuse it instead of making another API call
 - cache room booking suggestions so the same request doesn't trigger another API call
- Implement fallback logic (rule-based decisions) when API limits are reached
 - Used when AI fails or is rate-limited
- Use fallback logic (rule-based decisions if AI fails)
- Filter/clean inputs to reduce unnecessary API calls
- Batch or delay non-critical AI tasks (e.g., summaries) to reduce peak load and cost
 - ex) summaries processed later to reduce peak load

- Handle rate limits to avoid API throttling
 - Use request throttling and retry strategies

Result: Reduces cost and avoids API rate limits under heavy usage.

6. Asynchronous Processing

Heavy tasks (AI analysis, workflow processing) are not handled in real-time:

- Use background processing via n8n
- Return fast responses to users while processing continues

Result: Improves user experience and prevents system slowdown.

7. Monitoring and Observability

Use basic monitoring and alerts to maintain system performance:

- Track key metrics: request latency, error rate, CPU/memory usage, database connections, workflow failures
- Use centralized logging for API errors, database issues, and workflow failures
- Set alert thresholds (e.g., high latency, high error rate) to detect issues early

Result: Detects performance issues early and supports reliable scaling.

8. Multi-User / Multi-Agent Scalability

- Design system to handle multiple users and concurrent email requests
- Support multiple agents (AI/workflows) processing emails in parallel

Result: System can handle many users and emails at the same time without collisions or delays.

Conclusion

By using horizontal scaling, asynchronous processing, and efficient resource management, the system can handle up to 100,000 concurrent users while maintaining performance, reliability, and cost efficiency.